

Övning 11: Bygg en React-klient till ditt Movie API

Skapa ett användargränssnitt som interagerar med din C#-backend

I den här uppgiften ska ni knyta ihop säcken och bygga ett användargränssnitt i React som kommunicerar med det C# Web API ni byggde i Övning 3 - Movie WebAPI.

Gör så långt ni hinner och kan. Uppgiften är uppdelad i fyra nivåer som gradvis ökar i svårighetsgrad, så ta ett steg i taget. Vi rekommenderar att ni sätter upp ert projekt med ett modernt byggverktyg som Vite.

Nivå 1: Grunderna (Full CRUD)

Fokus: useEffect, datahämtning, formulärhantering och CRUD.

Ditt första mål är att bygga en applikation som kan visa, lägga till, uppdatera och ta bort filmer i databasen. Allt detta kan göras på en och samma sida (Startsidan).

- **Läs (GET):** Hämta alla filmer från `GET /api/movies` när komponenten laddas och visa dem i en lista, tabell eller som snygga kort.
- **Skapa (POST):** Bygg ett formulär med fält för *Titel*, *År*, *Genre* och *Speltid*. När formuläret skickas in ska datan postas till `POST /api/movies`. Uppdatera listan så den nya filmen syns direkt.
- **Uppdatera (PUT):** Lägg till en "Redigera"-knapp på varje film. Klickar man på den ska filmens data fyllas i formuläret. När man sparar ska en `PUT /api/movies/{id}` skickas för att uppdatera filmen i databasen och listan.
- **Radera (DELETE):** Varje film i listan ska ha en "Ta bort"-knapp. När man klickar på den ska `DELETE /api/movies/{id}` anropas och filmen ska försvinna från skärmen.

Nivå 2: Detaljvy och Recensioner

Fokus: React Router, dynamiska URL:er och relationer (1:M).

Nu ska vi göra applikationen till en riktig Single Page Application genom att lägga till flera sidor och låta användaren interagera med relaterad data.

- **Navigering:** Installera och konfigurera React Router. Gör så att man kan klicka på en film på startsidan för att navigera till en ny rutt, till exempel `/movies/:id`.
- **Detaljvy (GET):** På detaljsidan, hämta och visa all information om filmen via `GET /api/movies/{id}/details`. Detta inkluderar synopsis, budget och språk. Lista även upp alla skådespelare och recensioner som hör till filmen.
- **Skriv recension (POST):** Längst ner på detaljsidan ska det finnas ett formulär där användaren kan skriva in sitt namn, ett betyg (1-5) och en kommentar. Skicka detta till `POST /api/movies/{movieId}/reviews` och se till att recensionen dyker upp i listan direkt.

Nivå 3: Filtrering och Komplexa Relationer

Fokus: URL-parametrar (Query Strings) och många-till-många-relationer (N:M).

Nu är det dags att göra applikationen mer användarvänlig och bygga ut funktionaliteten med kopplingar mellan tabellerna.

- **Filtrera filmer:** Lägg till ett sökfält och en dropdown för genrer på startsidan. När användaren söker ska du anropa API:et med Query-parametrar, till exempel `GET /api/movies?genre=Action` och uppdatera listan. (Observera att detta kräver att du har byggt in stöd för filtrering i ditt API).
- **Hantera Skådespelare (N:M):** Inne på en films detaljsida, bygg funktionalitet för att lägga till befintliga skådespelare i filmen.
 1. Hämta alla skådespelare i databasen via `GET /api/actors` och fyll en dropdown-menyn (`<select>`).
 2. När användaren väljer en skådespelare, fyller i en roll (t.ex. "Huvudskurk") och klickar på spara, anropar du `POST /api/movies/{movieId}/actors/{actorId}`.

Nivå 4: Admin Dashboard

Fokus: Datapresentation, array-metoder och tredjepartsbibliotek.

Webbplatser behöver ofta en admin-vy för att sammanställa data. Bygg en helt ny vy (/dashboard) som bara fokuserar på statistik.

- Använd din `ReportsController` i API:et för att hämta statistik.
- Visa "Topp 5 filmer per genre", "Genomsnittsbetyg" och "Mest aktiva skådespelare".
- **Utmaning:** Installera ett bibliotek som *Recharts* (eller liknande) och försök visualisera datan som grafiska stapel- eller cirkeldiagram.